System Bug Report

Laravel Institute Management System — Backend (Laravel) & Mobile App (Flutter)

Project	d:\xampp\htdocs\laravel_institute
Scope	Multi-tenant authorization, fees/billing, attendance/exam/notifications, mobile app core layers, and the currently uncommitted work-in-progress diff
Method	Static code review (parallel targeted audits by subsystem) — no dynamic/penetration testing performed
Date	2026-07-18
Branch	main (with local uncommitted changes)

Severity	Count	Meaning
CRITICAL	8	Exploitable now for financial fraud or cross-tenant/cross-account data exposure; no special access required beyond a valid login
HIGH	12	Real data-integrity, correctness, or security bugs with clear impact but narrower conditions
MEDIUM	9	Real bugs with limited blast radius, or degraded UX/reliability
LOW	3	Minor correctness/hygiene issues, low impact

How this report was produced: given the size of the codebase (141 controllers, 120 models, plus a separate Flutter app), four targeted audits were run in parallel over the highest-risk areas — multi-tenant authorization, the fees/billing module, attendance/exam/notifications, and the mobile app's network/auth/state layers — plus a direct review of the changes currently sitting uncommitted in the working tree. This is **not** an exhaustive line-by-line audit of all 141 controllers; it is a risk-prioritized sweep. Treat the Critical items below as an immediate action list.

1. Multi-Tenant Security & Authorization (9 findings)

This app is multi-school: every record should be scoped to the caller's `school_id`. The reference pattern used correctly elsewhere is `$schoolId = $request->attributes->get('current_school_id') ?? $user->primarySchool()?->id ?? $user->firstTeacherSchoolId();`, then filtering queries by it (e.g. `LessonEvaluationController` does this correctly throughout). The billing module and a few other endpoints do not follow this pattern.

CRITICAL Payment creation endpoint has no auth-role check and no school scoping

`app/Http/Controllers/Billing/PaymentController.php:12 — POST /api/v1/billing/payments`

Bug: No role middleware restricts who can call this. `student_id` and role are attacker-controlled from the request body. The created `Payment` row doesn't even set `school_id`, so it's invisible to school-scoped reports afterward.

Impact: Any authenticated user (e.g. a parent account) can forge a "settled" payment and receive a real receipt for an arbitrary student, without an admin/cashier ever approving it.

CRITICAL Fee collection endpoint lets non-teacher roles pay for arbitrary students

`app/Http/Controllers/Api/FeeCollectionController.php:109 — collectFees(), POST /api/v1/billing/fees/collect`

Bug: No role middleware on the route. The ownership check ("is this student assigned to this teacher") only executes when `$user`'s role is literally `teacher` — every other role skips the check entirely.

Impact: A parent or student account can mark any other student's fees as paid.

CRITICAL Payment receipts readable across schools/students (IDOR)

`app/Http/Controllers/Billing/ReceiptController.php:17 — show()/downloadPdf()`

Bug: Receipt fetched by raw numeric ID with no ownership or school check, and no role middleware on the route.

Impact: Any logged-in user can enumerate receipt IDs and read/download other families' payment receipts (names, amounts, fee breakdowns).

CRITICAL Student dues and fee statements readable across schools (IDOR)

`app/Http/Controllers/Billing/DueController.php:11, StatementController.php:12, and Api/FeeCollectionController.php:24 (getDueFees)`

Bug: All three fetch by raw `{student}` ID with no ownership/school check. `getDueFees` additionally leaks a raw `_debug` block in the API response.

Impact: Enumerate student IDs to read any student's outstanding dues and full fee statement across the whole platform, not just one school.

HIGH Cashier deposit-acceptance endpoint missing school filter

`app/Http/Controllers/Api/CashierManagementController.php:225 — acceptDeposit()`

Bug: The `teacher_deposits` row is fetched without a school-scope filter.

Impact: A cashier/admin at School A could accept/manipulate a cash-deposit record belonging to School B.

HIGH Notice detail endpoint missing tenant/role check

`app/Http/Controllers/Api/NoticeController.php:120 — show()`

Bug: Unlike every sibling method in the same controller, `show()` has zero tenant or role check.

Impact: Cross-school notice content disclosure by ID enumeration.

CRITICAL Parent-child resolution matches by phone number only, no school scoping

`app/Http/Controllers/Api/ParentController.php:881 — resolveChildren()`

Bug: Children are resolved via `guardian_phone` match with no `school_id` constraint. This helper feeds nearly every parent-facing endpoint (attendance, fees, exam marksheets, notices).

Impact: If a phone number is reused across two schools' student records (common with siblings, phone number recycling, or families with children at different institute branches), a parent at School A can end up viewing a child's full academic/financial record at School B. This is the highest-leverage bug in the report because it's a single root-cause function that fans out into many parent-facing screens.

CRITICAL Unpublished / partially-graded exam results visible to parents

`app/Http/Controllers/Api/ParentController.php:427-520 — examResults()`

Bug: Results are computed live via `ResultCalculationTrait` and gated only on `Exam.status` (draft/active/completed/cancelled — a routine workflow field set in `Principal\ExamController.php:105`), never on `Result.is_published`, which is the field the actual "Publish Results" action (`Principal\ResultController.php:1319-1336`) sets.

Impact: The moment a principal marks an exam "completed" — often done right after the exam period ends, before all marks are entered — parents can immediately see full computed grades/GPA, even if a principal explicitly clicked "unpublish" or never published at all.

HIGH Mobile mark-entry endpoint doesn't validate marks against subject max

`app/Http/Controllers/Api/TeacherExamController.php:533-542 — saveMark()`

Bug: Validation is `nullable|numeric` with no `max:` rule, unlike the web equivalent (`Principal\MarkEntryController.php:108-110`) which correctly caps against `creative_full_mark` etc.

Impact: A teacher on the mobile app can submit e.g. 999 marks for a 30-mark subject. Saved as-is, it corrupts totals/GPA downstream and is visible to parents per the previous finding.

2. Fees & Billing (6 findings)

HIGH Idempotency key captured but never enforced — double payment on retry

`app/Http/Controllers/Api/FeeCollectionController.php:145 — collectFees()`

Bug: An `X-Idempotency-Key` header is read but never checked before creating a Payment. `StudentFee::lockForUpdate()` only serializes concurrent writes to that row — it doesn't deduplicate a retried request.

Impact: A double-tap on the mobile "pay" button, or a network-timeout-triggered client retry, creates two full Payment + PaymentItem + Ledger writes for the same fee.

HIGH Online payment can be double-settled via race between callbacks

`app/Http/Controllers/Billing/SSLCommerzCallbackController.php:30 — success() / ipn()`

Bug: Both the browser-redirect callback and the server-to-server IPN callback independently do a check-then-write (`status !== 'settled'` then `settlePayment()`) with no locking between them. These two callbacks commonly fire concurrently for the same transaction.

Impact: The same online payment can be settled/recorded twice.

CRITICAL Payment can be applied to another student's fee record

`app/Http/Controllers/Api/FeeCollectionController.php:166 — collectFees()`

Bug: `student_fee_id` is validated only with `exists:student_fees,id` — never checked against the requesting `student_id` / `school_id`.

Impact: Combined with the missing role check in finding 1.2, a payment request can settle a completely different student's (or a different school's) fee record.

HIGH Overpayment silently absorbed with no cap or credit tracking

`app/Http/Controllers/Api/FeeCollectionController.php:117 — collectFees()`

Bug: `fees.*.amount` is never capped against the fee's actual remaining due.

Impact: Overpayment is accepted and merged straight into `paid_amount` with no refund/credit-balance record — accounting can't reconcile it later.

MEDIUM Direct payment creation has no transaction wrapping the receipt step

`app/Http/Controllers/Billing/PaymentController.php:12 — store()`

Bug: Receipt issuance after payment creation isn't wrapped in a DB transaction or try/catch.

Impact: A failure between the two steps leaves an orphaned "settled" payment with no receipt ever generated.

MEDIUM **Partial fee waiver can leave a fee's status permanently stale**

`app/Http/Controllers/Api/FeeWaiverController.php:193 — applyWaiverToExistingFees()`

Bug: Status only flips to `paid` when the waived amount is `<= 0`. If a partial waiver drops the remaining amount below what's already been paid, status stays at its prior value (e.g. `partial`).

Impact: Due/report screens that filter directly on `status` keep showing this fee as unpaid/partial indefinitely, even though it's effectively settled.

3. Attendance, Exams & Notifications (6 findings)

HIGH **GPA calculation counts ungraded subjects as failures**

`app/Traits/ResultCalculationTrait.php:345, 368-374`

Bug: When a subject has no mark record yet, grade is set to `N/R` with 0 grade points, but it's still counted into both `subjectCount` and `failedSubjectCount`. This recalculation runs after *every single subject mark save* (`MarkEntryController.php:149`, `TeacherExamController.php:593`).

Impact: A student with only 1 of 5 subjects graded is computed as GPA≈0 / letter grade F, not "incomplete." Combined with the unpublished-results leak above, parents can see their child as "failing" mid-grading.

HIGH **Auto-absent job ignores holidays and weekly off-days**

`app/Console/Commands/ProcessEndOfDayAttendance.php:74-113 — markAbsentStudents(), scheduled every 5 minutes`

Bug: Every enrolled student without an attendance record for "today" is marked `absent`, with no check against the `Holiday` / `WeeklyHoliday` models that exist and are used elsewhere in the codebase, and no weekday guard on the schedule itself.

Impact: On Fridays/Saturdays or any declared holiday, every student in every school gets bulk-marked absent, and guardians receive "your child was absent today" push notifications on days school wasn't in session.

MEDIUM **Guardians can receive duplicate attendance notifications**

`app/Console/Commands/ProcessEndOfDayAttendance.php:118-145 — sendEndOfDayNotifications()`

Bug: This end-of-day job pushes a notification for every attendance record without checking whether that record was already notified when the teacher submitted attendance in real time (`TeacherStudentAttendanceController.php:332-337` already sends one). The only dedup guard is once-per-school-per-day, which stops the job re-running, not the double notification.

Impact: Guardians get two separate push notifications for the same attendance status on the same day.

HIGH **Biometric punch processing race condition with no failure handling**

`app/Services/Biometric/AttendanceProcessingEngine.php:155-226 & 242-283; app/Jobs/ProcessBiometricPunchJob.php:29-32`

Bug: Non-atomic check-then-insert (query for existing record, then create) with no transaction or row lock. The table has a unique constraint, so concurrent punch-processing throws a `QueryException` on the second insert — and the job has no try/catch or `failed()` handler.

Impact: The punch is silently lost, and the `BiometricAttendanceLog.sync_status` stays stuck at `pending` forever, permanently skewing the admin dashboard's pending/failed counts.

MEDIUM **Dead FCM device tokens are never cleaned up**

`app/Services/FcmService.php:59-85 — sendToToken()`

Bug: On delivery failure (including FCM's explicit "unregistered/invalid token" errors) the method only logs to `NotificationLog` — it never deletes/deactivates the `DeviceToken` row, and no cleanup exists anywhere else.

Impact: Every notice, homework, attendance, and duty notification keeps re-attempting delivery to the same dead tokens indefinitely — wasted FCM calls and an ever-growing log table.

LOW **Biometric notification job computes a specific message but never sends it**

`app/Jobs/SendAttendanceNotificationJob.php:48-70`

Bug: The job builds an entry/exit-specific title and body (with punch time) but then calls `sendAttendanceNotification()` with just status/date/type — which internally rebuilds a generic message. The `'biometric'` type isn't special-cased in that method's switch, so it falls into the generic branch.

Impact: Guardians never see the entry/exit time in biometric attendance notifications; the carefully constructed message is dead code.

4. Mobile App (Flutter) (9 findings)

HIGH **Auth token stored in plaintext SharedPreferences, not secure storage**

`mobile_app/lib/core/network/dio_client.dart:34-35; mobile_app/lib/data/auth/auth_repository.dart:71-72`

Bug: No use of `flutter_secure_storage` (or Android Keystore-backed storage) anywhere in the auth path — the bearer token sits in an unencrypted SharedPreferences XML file.

Impact: Readable via Android backup extraction or on a rooted/compromised device without needing the app's own vulnerabilities.

HIGH Auth tokens and full API payloads logged in plaintext

`mobile_app/lib/core/network/dio_client.dart:45-53;`
`mobile_app/lib/data/auth/auth_repository.dart:32-39`

Bug: Every response body is logged via `developer.log`, including an explicit "RAW RESPONSE DATA" log of the login response (which contains the bearer token). Not stripped from release builds.

Impact: Tokens and personal data recoverable from device logs via `adb logcat` on any device with USB debugging enabled.

HIGH No global handling for expired/invalid sessions (401)

`mobile_app/lib/core/network/dio_client.dart:31-64`

Bug: The Dio error interceptor only logs errors and forwards them — nowhere in the app is a 401 response used to force logout / redirect to the login screen (verified: zero matches app-wide).

Impact: Once a token expires or is revoked server-side, every screen just shows a generic error forever; there's no path back to login without the user manually finding the logout button.

MEDIUM Logout doesn't invalidate cached account-scoped data providers

`mobile_app/lib/presentation/state/auth_state.dart:52-57 — AuthNotifier.logout()`

Bug: Logout only clears the auth token and one cached-profile key; it never invalidates the Riverpod `parent*Provider` / `selectedStudentIdProvider` family. A separate "Reload" button elsewhere in the app does invalidate these, confirming this is a gap rather than intentional.

Impact: If a second account logs in on the same device without the app being killed, stale cached data from the previous account (selected child, fees, homework) can still be served under the new session.

MEDIUM Null-safety crash when both section-list requests fail

`mobile_app/lib/data/teacher/teacher_students_repository.dart:293 — fetchSections()`

Bug: `_classScopedCache![classId]['sections']` indexes without a null-safe `?[classId]`. If the primary and fallback endpoints both fail (offline/timeout), the cache entry for that class was never set. Compare to the sibling `fetchGroupsForClass`, which correctly uses `?[classId]`.

Impact: Throws `NoSuchMethodError` on `null` instead of surfacing a clean network-error message.

MEDIUM setState called after dispose in a paginated list screen

`mobile_app/lib/presentation/features/teacher/teacher_students_list_page.dart:74, 153, 164`

Bug: `setState()` runs immediately after an `await` network call with no `mounted` guard (only the `finally` block checks it).

Impact: Navigating back while a page/meta fetch is still in flight crashes with "setState() called after dispose()".

MEDIUM Same setState-after-dispose bug in a widely-reused dashboard widget

`mobile_app/lib/widgets/animated_tile.dart:54, 56 — _loadRive()`

Bug: After `await RiveFile.asset(...)`, both the success and catch paths call `setState()` unconditionally.

Impact: Because this tile is used broadly across dashboards, any navigation/tab-switch away while the animation asset is still loading can crash the app.

MEDIUM App-wide cleartext (HTTP) traffic permitted with no scoping

`mobile_app/android/app/src/main/AndroidManifest.xml:22`

Bug: `android:usesCleartextTraffic="true"` is set globally with no `network_security_config.xml` restricting it to specific dev hosts.

Impact: Combined with the plaintext-logging bug above and the env.dart issue below, any accidental non-HTTPS production config would transmit tokens/credentials unencrypted with no OS-level guard.

LOW Unguarded type cast on API response bodies

`mobile_app/lib/data/notice/notice_repository.dart:17;`
`mobile_app/lib/data/parent/parent_repository.dart:101, 123`

Bug: `resp.data['data'] as Map<String, dynamic>` with no type guard, unlike `_parseList()` in the same files which checks defensively.

Impact: Any error/validation response shaped differently than expected throws an unhandled `TypeError` instead of a catchable, user-facing error message.

5. Uncommitted Work-in-Progress (current working tree)

The lesson-evaluation / homework-notification feature currently being built (uncommitted changes to `LessonEvaluationController.php`, `PushNotificationService.php`, `lesson_evaluation_mark_page.dart`, `env.dart`, `routes/console.php`) is otherwise sound — the new attendance-lock and homework-pairing logic is correctly scoped and transactional. Two issues should be fixed before merging:

CRITICAL (RELEASE-BLOCKER) Default API base URL points at a local emulator address, not production

`mobile_app/lib/core/config/env.dart:5`

Bug: The default value was changed from `https://institute.batighorbd.com/api/v1/` to `http://10.0.2.2/laravel_institute/public/api/v1/` (the Android-emulator alias for the host machine's localhost).

Impact: Any build produced without explicitly passing `--dart-define=API_BASE_URL=...` — including, easily, an accidental release build — will try to reach a local dev server that doesn't exist on real devices, and the app will fail to connect entirely.

Fix: revert the default back to the production URL, and instead pass the local URL via `--dart-define` only in local dev run configs (e.g. VS Code launch.json / a dev script), never as the compiled-in default.

LOW TextEditingControllers created per dialog open are never disposed

`mobile_app/lib/presentation/features/teacher/lesson_evaluation_mark_page.dart — _openSubmitDialog()`

Bug: The new submit-confirmation dialog creates `previousCtrl` and `nextCtrl` (both `TextEditingController`) locally on every call to `_openSubmitDialog()`, but neither is disposed when the dialog closes (confirmed or cancelled). The previous single class-level `_notesController` this replaced was properly disposed in `State.dispose()`.

Impact: A small memory leak on every dialog open; a teacher opening/cancelling the dialog repeatedly (e.g. correcting a typo, backing out) accumulates undisposed controllers for the life of the page.

Fix: wrap the controller creation/dialog/dispose in a try/finally, or call `previousCtrl.dispose(); nextCtrl.dispose();` right after `showDialog` resolves.

Generated by static code review — recommend confirming each Critical finding with a quick manual reproduction before prioritizing fixes, and re-running route/middleware checks after any auth refactor.